

Design Pattern – Factory Method

Adam HAMOUC

May 2026

1. Intention

Définit une interface pour créer un objet, mais **délègue aux sous-classes** le choix du type concret à instancier.

Le code client manipule uniquement l'interface parente — il ne sait jamais quel objet concret il reçoit.

2. Structure

Deux hiérarchies parallèles, chacune basée sur l'héritage :

- **Produits** : classe abstraite (interface) + classes concrètes qui l'implémentent.
- **Fabriques** : classe abstraite avec une méthode virtuelle pure `create()`, sous-classes qui la définissent et instancient le produit concret.

La fabrique concrète décide du type instancié. Le client passe uniquement par les interfaces abstraites des deux côtés.

3. Exemple concret (moteur de rendu)

```
// Produit abstrait
class RenderComponent {
public:
    virtual void draw(const Scene& scene) = 0;
    virtual ~RenderComponent() = default;
};

// Produits concrets
class VulkanRenderer : public RenderComponent {
public:
    void draw(const Scene& scene) override { /* pipeline Vulkan */ }
};
class DX12Renderer : public RenderComponent {
public:
    void draw(const Scene& scene) override { /* pipeline DX12 */ }
};

// Fabrique abstraite
class RendererFactory {
public:
    virtual std::unique_ptr<RenderComponent> create() = 0;
    virtual ~RendererFactory() = default;
};
```

```
};

// Fabriques concretes
class VulkanFactory : public RendererFactory {
public:
    std::unique_ptr<RenderComponent> create() override
    { return std::make_unique<VulkanRenderer>(); }
};
class DX12Factory : public RendererFactory {
public:
    std::unique_ptr<RenderComponent> create() override
    { return std::make_unique<DX12Renderer>(); }
};

// Code client : ne connait que les interfaces abstraites
void GameEngine::init(RendererFactory& factory) {
    renderer = factory.create(); // Vulkan ou DX12 ? peu importe
    renderer->draw(scene);
}
```

4. Cas d'usage (engine / game dev)

- **Multi-backend de rendu** : swaper Vulkan / DX12 / Metal sans toucher au code client.
- **Spawner d'entités** : factory d'ennemis, de projectiles, de power-ups selon le niveau.
- **Asset loading** : factory de textures, meshes, shaders selon la plateforme cible.
- **Composants ECS** : factory de composants (Physics, Audio, AI) enregistrés dynamiquement.

5. Avantages / Inconvénients

Avantages :

- **Scalable** : ajouter un type = ajouter une sous-classe, sans modifier l'existant (Open/Closed).
- Élimine les if/switch de construction éparpillés dans le code.
- Le client ne dépend jamais des types concrets — isole les dépendances.
- Facilite les tests : on peut injecter une fabrique mock.

Inconvénients :

- Multiplication des classes : chaque nouveau type requiert une fabrique dédiée.
- Peut être over-engineering si les types concrets sont peu nombreux et stables.

6. Sources

<https://refactoring.guru/fr/design-patterns/factory-method>